

How a company runs on FallForge

Local-first by construction. Frontier only when the local model genuinely can't. Every step — what ran, what it cost, what it answered — lands on a hash-chained, re-verifiable record.

FallForge mints a company its own task-specific AI workers, proves each one beats a baseline before it's trusted, serves it on the company's own hardware behind real capability grants, and only reaches past the local model when a deterministic check says it must. A separate, proven assembly layer (fallbrain) runs the company day to day on top of it: it decides what needs doing next by a fixed priority law, and it never lets an AI turn a key at the four doors — money, legal, taste, client-trust — that stay human by design. This brief shows the real, gated mechanism end to end, then a worked example with real numbers.

The request path, end to end

■ a step that runs ■ a decision gate ■ a human door

1 A task arrives

e.g. "triage this ticket", "classify this document" — one of the repetitive, high-volume jobs a company was renting a frontier model to do per token.

2 The company's own minted model answers first — on its own hardware

Not a shared generic model: a small model *fallforgemint* tuned to this exact task, and gated at mint time — it only ever got minted because it measurably beat its raw base on held-out examples. Charged 1 unit against the holder's grant. *fallnode kernel.mjs* — *CHARGE_ASK*

3 Did it answer in the required shape?

A deterministic, mechanical check — required JSON fields present, format valid. Not "is it a good answer": that's the mint-time gate's job. This check only asks whether the shape held. *fallnode kernel.mjs* — *shouldEscalate()*

4 Yes → done. No → escalate to the limb, labelled and priced higher

Only a *format failure* escalates — never "the small model might be wrong." The bigger model is a tool the small one borrows, not the default. The answer is labelled `escalated: true, by: <limb>` and charged 5 units, not 1 — the cost difference is visible, not hidden. *fallnode kernel.mjs* — *CHARGE_LIMB*

5 Does the outcome need a human key?

Money moving, a legal commitment, a taste/publish call, or anything touching client trust always queues at its door. No grant, however large, buys past a door. *fallbrain brain.mjs* — *tick()*

6 Every step lands on the record

The ask, the escalation (if any), the door decision (if any) — hash-chained into a service ledger, and bound to the model's own signed, gated manifest through the provenance chain. Re-verifiable offline by anyone, not asserted. *fallnode kernel.mjs* — *appendLedger()* · *fallforge kernel.mjs* — *provenanceChain()*

The sovereign-first cascade

"Local-first, frontier only when needed" is not a policy on this system — it's a cost and a gate, enforced in code that's already shipped and mutation-tested.

WHY THE SMALL MODEL IS THE DEFAULT, NOT A FALLBACK

A model only gets served at all once it has cleared fallforgemint's gate: measured against its own raw base, on the operator's own held-out examples, and only minted on a certified win. So "local-first" isn't "cheap and hopeful" — the thing running by default has already beaten the thing it's not renting.

THE WALLET RULE

Every action is checked against a grant *before* it runs — capability and budget both, not after the fact. A refusal costs nothing. Budget zero is a full stop, not a slow-down.

```
capability check  action's organ ∈
                  grant.capabilities
budget check     grant.budget > 0
on either failing REFUSED, charged 0
fallbrain brain.mjs — tick(state, grant)
```

THE ESCALATION, PRICED HONESTLY

A shipped, real session: `triage-1b` served with `qwen2.5:7b` as the limb. A "forget the form" attack still held format (charged 1). A YAML+French attack broke the small model's format — the limb answered, labelled, charged 6. A forged token died on the signature — charged 0.

fallnode — the shipped session, re-verified in-browser

THE SAME PATTERN, PROVEN A SECOND WAY ELSEWHERE IN THE ESTATE

A separate, already-shipped router (`si-didy-cascade`) classifies each request into a tier before it runs: **T0** mechanical/deterministic ops (zero cost) → **T2** a browser-local model → **T2.5** the operator's own Ollama model, tried before anything paid leaves the machine → **T3** the frontier model, only when nothing local could answer. That repo states a 60–80% token reduction from this routing — its own figure, not independently re-measured for this brief, so treat it as a claim to verify rather than a proven number. It's cited here as a second, separately-built example of the same sovereign-first shape, not as the same code path as the FallForge stack above.

The rule that actually enforces "frontier only when needed" is narrow on purpose: escalation triggers on a *deterministic shape failure*, never on a model's own uncertainty about whether it's right. Quality is the minting gate's job, done once, before the model is ever served. The runtime's job is narrower and provable: did the required fields come back at all.

PART 2

The trust stack a company's AI workers run on

Nine stages, each an independent, self-contained product with its own mutation gate — composed by content hash, not merged into one codebase. A company doesn't adopt one monolith; it wires proven, separately-gated organs together.

STAGE	DOES	REAL REPO · FUNCTION	STATUS
Mint	Turns a task + a few examples into an owned, signed model — measured against its base, never asserted better	<code>fallforge</code> mint — scorecardReceipt, makeManifest	LIVE
Prove	The scorecard is the proof: held-out hits vs. base hits, verdict BEATS/LOSES/TIES — a receipt allowed to say the model lost	<code>fallforge</code> mint — scorecard()	LIVE
Compose	Cross-checks independent nodes on one task per field, refuses a panel of clones — agreement without independence proves nothing	<code>fallforge</code> cell (Veridia) — adjudicate()	LIVE
Verify	Chains cross-checks into a verified process; aggregates many verified units into one department with a declared charter	<code>fallforge</code> cell — composePipeline(), composeDepartment()	LIVE
Audit	Independently re-checks a department against the real receipts it claims to summarise — not the department's own word for it	<code>fallforge</code> cell — auditDepartment()	LIVE
Serve	Runs the minted model on the company's own hardware behind capability grants; ask/limb escalation; hash-chained ledger	<code>fall</code> node — server.mjs, charge(), appendLedger()	LIVE
Catalogue	The shelf: a node is listed only if it certifiably beat its base; own-vs-rent on the buyer's real rates, can say rent	<code>fallforge</code> -catalogue — validListing(), savings()	LIVE
Compliance	A heuristic pre-screen against the Act's own categories; a factual reading of what's actually published, not asserted	<code>fallforge</code> — riskSelfCheck(), compliancePosture()	LIVE
Provenance	Walks the hash a downstream artifact declares about its neighbour and says exactly where the chain breaks, if it does	<code>fallforge</code> — provenanceChain()	LIVE

COMPOSED BY REFERENCE, NOT BY MERGER

Each stage stays an independent, self-contained page with its own gate. The hub doesn't import their code — it takes the hash each stage already produces and checks the next stage actually points at it: does the shelf listing carry *this* mint's manifest hash; does the running server's ledger carry *this* manifest hash; does an audit's real receipt hash sit inside the department it claims to belong to. A department can be minted and shelved with nothing downstream yet — that's an honest, named state (`reach: [shelved]`), not a failure. A mismatch anywhere is named at the exact link, never silently passed.

This is the mechanism that makes "the whole thing is auditable end to end" a checkable fact rather than a claim: run `verifyProvenanceReceipt()` on the chain yourself and it recomputes the same verdict from the same hashes, offline, no server involved.

A day in a running company

The FallForge stack mints and serves the AI workers. A separate, already-proven assembly layer decides what the company does with them, minute to minute — and never lets an AI decide anything at the four doors.

THE PRIORITY LAW (FIXED ORDER, ARGUED, NOT ARBITRARY)

#	TRIGGER	ROUTE
1	A statutory cooling-off period expired	legal door — a right is live now
2	An open complaint	client-trust door — the regulatory clock is running
3	Pending client due-diligence (AML)	legal door — the firm must not act for an unverified client
4	A periodic KYC review is due	auto-drafted
5	An unpaid invoice	auto-drafted (chaser only — moving money is a door)
6	Documents awaited	auto-generated via the paper organ
7	Nothing urgent	the day is summarised into memory

fallbrain brain.mjs — nextAction(state). Every decision states which rule it rests on; a decision that can't say why isn't made.

THE DOORS: MONEY, LEGAL, TASTE, CLIENT-TRUST — ALWAYS HUMAN

A door decision is never auto-run, no matter how large the grant. It enters a queue with its reason attached; a human turns the key with an argued reason (a one-word refusal is refused as a non-answer); a decided key stays decided — a second turn on the same key is refused, because an audit trail where verdicts move isn't a trail.

WHAT THE ORGANS ACTUALLY COMPUTE (REAL, GATED, NOT ILLUSTRATIVE PROSE)

An example of the case-work layer (`fallcorp-demo` — a full simulated day for a UK claims firm) already computes: a DSAR deadline from the date received; an 8-week statutory response deadline; invoice totals with VAT; a trial balance with FX applied; a settlement split to the exact penny; an expert-witness booking that *refuses on a genuine interval clash* rather than silently double-booking. Re-run today: its own test suite passes 6/6 against a near-miss gauntlet built to catch a guard that only looks like it's checking. None of this is worded by a model — it's arithmetic and calendar law.

Put together: the priority law decides *what* needs doing and *who* must approve it; the case-work organs compute the real facts (dates, totals, clashes); a FallForge-minted, proven model words the draft from those facts only, never inventing a figure; the human at the relevant door reads the draft against the real facts and turns the key. The model never gets to be the source of a fact — only of the wording.

Worked example: an illustrative SME

ILLUSTRATIVE — NOT A GUARANTEE. CHANGE THE ASSUMPTIONS, THE ANSWER CHANGES.

A small claims-handling team runs one repetitive task — ticket triage / document classification — through a FallForge-minted specialist instead of a rented frontier model. Figures below are computed by the estate's own live cost kernels, not typed by hand.

STATED ASSUMPTIONS		COMPUTED OUTPUT	
Team	15 people, claims handling	Tokens / month	12,000,000
Asks / month	20,000 (~60/person/working day)	Cloud cost / month	\$36.00
Tokens / ask	600	Local electricity / month	\$3.60
Cloud rate	\$3.00 / million tokens	Saved / month	\$32.40
Local energy	3 kWh / million tokens	Saved / year	\$388.80
Electricity	\$0.10 / kWh	Break-even	immediate (no mint fee)
Mint fee	\$0 (the free tier)	Verdict	CHEAPER-TO-OWN / OWN_WINS
Cloud rate is a stated illustrative figure, not a live quote — re-check current pricing before relying on it. The kWh/electricity defaults are the same ones fallforge-catalogue's own live calculator ships. All other inputs are set here and shown so the result can be checked or changed.		Computed live by fallforge-catalogue's savings() and fallforgemint's ownVsRent() — the same functions the public calculators run, not re-derived by hand.	

What this figure does **not** include: the SME's own hardware cost, setup and maintenance time, or staff time to review drafts at the doors. It compares only the marginal cost of running the tokens — cloud rental vs. local electricity — for one repetitive task at this volume. A calculator that can only ever say "buy" is a brochure; at low enough volume or high enough local energy cost, the honest answer from these same functions is **CHEAPER-TO-RENT** . Run your own numbers at the live calculator before deciding anything.

Honest scope: what's real today, what isn't built yet

Every stage on page 3 is live, gated, and independently checkable today. What does **not** yet exist is the single wired path where fallbrain's priority law hands a decision directly to a FallForge-minted worker and the result lands on the provenance chain automatically — today those are two proven systems, not yet one wired loop.

PIECE	STATUS	WHAT'S REAL ABOUT IT
The trust stack (mint→prove→compose→verify→audit→serve→catalogue→compliance→provenance)	LIVE	Nine independently gated products, each with a witness mutation gate; the provenance organ proven against real cross-repo artifacts, not fixtures
The ask→limb local-first escalation	LIVE	Shipped, gated, a real served session captured and re-verified in-browser
The company assembly & priority law (fallbrain)	LIVE	Five gated laws, proven end-to-end on real seeded records across ten estate verticals
A running company day (fallcorp-demo)	LIVE	A full simulated day, deterministic; its test suite re-run for this brief passes 6/6, real DSAR/invoice/escrow/clash-detection arithmetic
fallbrain deciding → a FallForge-minted worker acting → the provenance chain recording it, as ONE wired loop	NOT YET BUILT	This document is the architecture for it — the next phase, not a claim about today

Nothing above is invented for this document. Every figure, function name, and gate score is drawn from the repos as they actually stand — re-run any of the cited functions yourself and they reproduce the same numbers, because that is the entire point of gating them.

No pricing appears on this page by design — this describes what the architecture does, not what anything costs to buy. Compliance positioning stated elsewhere by AI-Native Solutions describes the architecture (GDPR by design, EU AI Act minimal-risk) and is not a third-party certification.